

Practical_ML_Project

Pratik A. Khare

2026-02-16

```
## Warning: package 'pander' was built under R version 4.5.2
```

```
## Warning: package 'caret' was built under R version 4.5.2
```

```
## Loading required package: ggplot2
```

```
## Loading required package: lattice
```

```
## Warning: package 'rpart.plot' was built under R version 4.5.2
```

```
## Loading required package: rpart
```

```
## Warning: package 'rpart' was built under R version 4.5.2
```

```
## Warning: package 'doParallel' was built under R version 4.5.2
```

```
## Loading required package: foreach
```

```
## Warning: package 'foreach' was built under R version 4.5.2
```

```
## Loading required package: iterators
```

```
## Warning: package 'iterators' was built under R version 4.5.2
```

```
## Loading required package: parallel
```

Objective of the project

The objective of this project is to develop and evaluate predictive models for classifying human activity using accelerometer data. Through exploratory data analysis, data preprocessing, and comparison of multiple classification algorithms, we aim to identify a robust and accurate model for reliable out-of-sample prediction.

Exploratory Data Analysis

Now, let's load the data and perform some exploratory data analysis

- The dimensions of the training dataset are 19622 by 160 and for the validation dataset are 20 by 160
- The classe variable which will be the output for our predictions looks like this:

A	B	C	D	E
5580	3797	3422	3216	3607

- The first 20 column names in training and test set are: X, user_name, raw_timestamp_part_1, raw_timestamp_part_2, cvtd_timestamp, new_window, num_window, roll_belt, pitch_belt, yaw_belt, total_accel_belt, kurtosis_roll_belt, kurtosis_pitch_belt, kurtosis_yaw_belt, skewness_roll_belt, skewness_pitch_belt, skewness_yaw_belt, max_roll_belt, max_pitch_belt, max_yaw_belt.
- We won't go into the details of other columns as there are many. Since we need to filter the empty or N/A data though, let's examine what percentage of the data is N/A: 41.0085618%

Data Filtering

Before applying any individual data filtering steps, we shall split the training dataset into training and testing:

```
set.seed(123)
inTrain <- createDataPartition(y=training_imported$classe, p=0.6, list=FALSE)
training <- training_imported[inTrain,]
test <- training_imported[-inTrain,]
```

1. Getting rid of variables which are not predictors, such as "username", "timestamp" etc:

```
cols_to_remove <- c("X", "user_name", "raw_timestamp_part_1", "raw_timestamp_part_2", "cvtd_time
stamp", "new_window", "num_window")
training_filtered <- training[, !(names(training) %in% cols_to_remove)]
```

2. Getting rid of all columns which have greater than 90% of the values as N/A:

```
na_cols <- colSums(is.na(training_filtered))>0.9*nrow(training)
training_filtered <- training_filtered[, !na_cols]
```

3. Removing variables with near zero variance which won't have much impact on the model:

```
nzv <- nearZeroVar(training_filtered)
training_filtered <- training_filtered[, -nzv]
```

Now, to ensure that the test and validation datasets have the same columns as the filtered training set:

```
test_filtered <- test[, colnames(training_filtered)]
val_filtered <- val_imported[, setdiff(colnames(training_filtered), "classe")]
```

Ultimately, we are left with 52 columns in the validation dataset and 53 columns in the training dataset

Model development and prediction

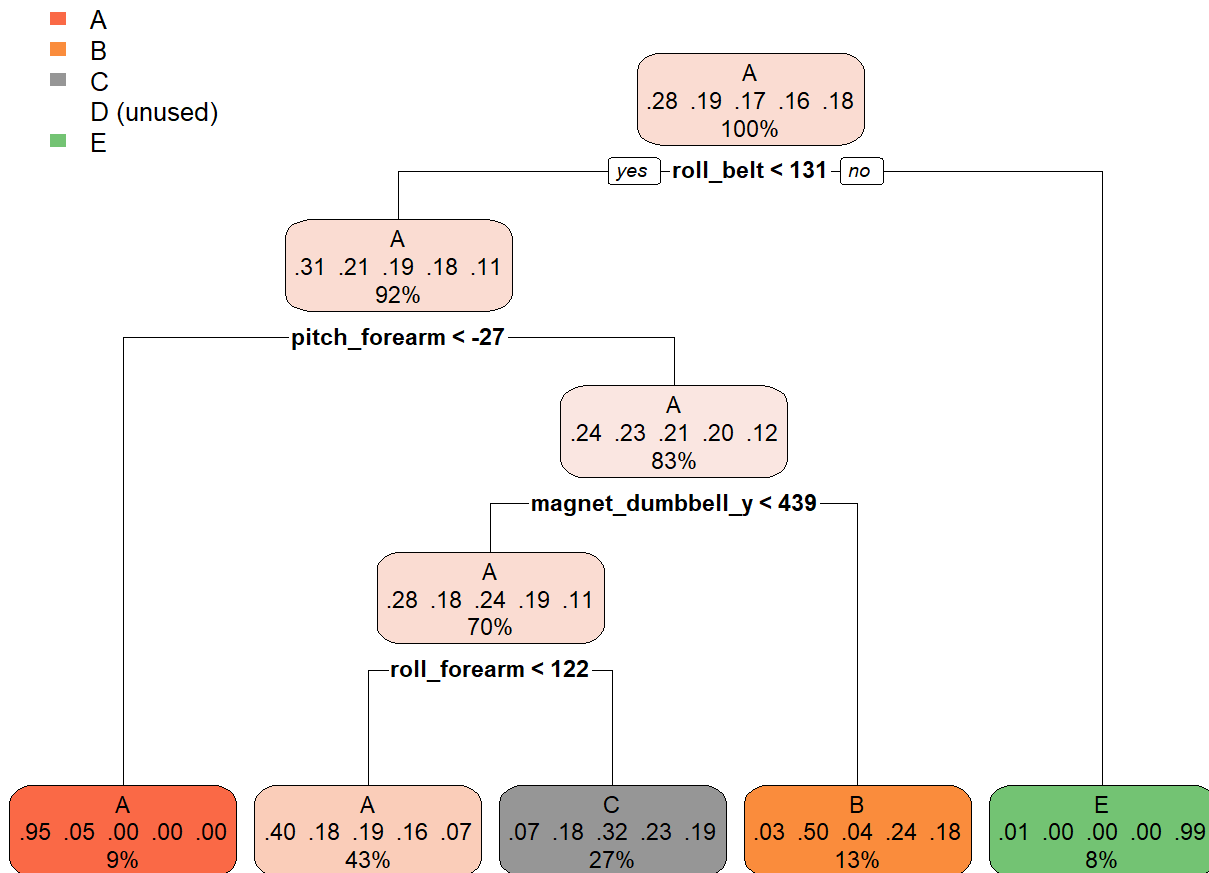
Next, we will use different classification algorithms and compare the accuracy of predictions on the test dataset. Specifically, we will use: 1. Decision Trees 2. Random Forest

We will begin by converting the classe variable to factor first:

```
# Let us ensure that the classe variable is a factor in the training and test data
training_filtered$classe <- factor(training_filtered$classe)
test_filtered$classe <- factor(test_filtered$classe, levels = levels(training_filtered$classe))
```

Decision Trees

```
ctrl <- trainControl(method = "cv", number = 5) # Cross validation parameter
moddt <- train(classe ~ ., data = training_filtered, method = "rpart", trControl = ctrl)
preddt <- predict(moddt, newdata = test_filtered[, setdiff(names(test_filtered), "classe")])
rpart.plot(moddt$finalModel)
```



```
cmdt <- confusionMatrix(preddt, test_filtered$classe)
cmdt
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction   A    B    C    D    E
##           A 2024  652  615  590  190
##           B   37  518   43  207  213
##           C  168  348  710  489  364
##           D    0    0    0    0    0
##           E    3    0    0    0  675
##
## Overall Statistics
##
##           Accuracy : 0.5005
##           95% CI : (0.4894, 0.5116)
##           No Information Rate : 0.2845
##           P-Value [Acc > NIR] : < 2.2e-16
##
##           Kappa : 0.3473
##
## McNemar's Test P-Value : NA
##
## Statistics by Class:
##
##           Class: A Class: B Class: C Class: D Class: E
## Sensitivity           0.9068  0.34124  0.51901  0.0000  0.46810
## Specificity           0.6354  0.92099  0.78867  1.0000  0.99953
## Pos Pred Value        0.4972  0.50884  0.34151    NaN  0.99558
## Neg Pred Value        0.9449  0.85354  0.88590  0.8361  0.89300
## Prevalence            0.2845  0.19347  0.17436  0.1639  0.18379
## Detection Rate        0.2580  0.06602  0.09049  0.0000  0.08603
## Detection Prevalence  0.5189  0.12975  0.26498  0.0000  0.08641
## Balanced Accuracy      0.7711  0.63111  0.65384  0.5000  0.73382
```

The accuracy of the decision tree **with 5-fold cross validation** is: 50.05%

Random Forest

```
cl <- makePSOCKcluster(2)
registerDoParallel(cl)
ctrl <- trainControl(method = "cv", number = 3, allowParallel = TRUE) # Cross validation parameter
modrf <- train(classe ~ ., data = training_filtered, method = "rf", trControl = ctrl, ntree = 100)
stopCluster(cl)
registerDoSEQ()

predrf <- predict(modrf, newdata = test_filtered[, setdiff(names(test_filtered), "classe")])

cmrf <- confusionMatrix(predrf, test_filtered$classe)
cmrf
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction   A    B    C    D    E
##           A 2225   13    0    0    0
##           B    5 1502   14    0    0
##           C    0    3 1349   14    2
##           D    0    0    5 1271    2
##           E    2    0    0    1 1438
##
## Overall Statistics
##
##           Accuracy : 0.9922
##           95% CI : (0.99, 0.994)
##           No Information Rate : 0.2845
##           P-Value [Acc > NIR] : < 2.2e-16
##
##           Kappa : 0.9902
##
##           McNemar's Test P-Value : NA
##
## Statistics by Class:
##
##           Class: A Class: B Class: C Class: D Class: E
## Sensitivity      0.9969  0.9895  0.9861  0.9883  0.9972
## Specificity      0.9977  0.9970  0.9971  0.9989  0.9995
## Pos Pred Value   0.9942  0.9875  0.9861  0.9945  0.9979
## Neg Pred Value   0.9988  0.9975  0.9971  0.9977  0.9994
## Prevalence       0.2845  0.1935  0.1744  0.1639  0.1838
## Detection Rate   0.2836  0.1914  0.1719  0.1620  0.1833
## Detection Prevalence 0.2852  0.1939  0.1744  0.1629  0.1837
## Balanced Accuracy 0.9973  0.9932  0.9916  0.9936  0.9984
```

The accuracy of the random forest **with 5-fold cross validation** is: 99.22%

Applying the model for the prediction assignment

```
final_ans <- predict(modrf, newdata = val_filtered)
```

The final predictions for the 20 unknown cases are: B, A, B, A, A, E, D, B, A, A, B, C, B, A, E, E, A, B, B, B

Conclusion

The random forest model is better as it predicts with a much higher accuracy and yields the correct predictions for the validation dataset (20 samples)